

Entrada del compiador:

// --- Utilidad global ---

```
function toString(x: integer): string {  
    return "";  
}
```

// --- Clase base ---

```
class Persona {  
    let nombre: string;  
    let edad: integer;  
    let color: string;
```

```
function constructor(nombre: string, edad: integer) {  
    this.nombre = nombre;  
    this.edad = edad;  
    this.color = "rojo";  
}
```

```
function saludar(): string {  
    return "Hola, mi nombre es " + this.nombre;  
}
```

```
function incrementarEdad(anos: integer): string {
```

```
    this.edad = this.edad + anos;

    return "Ahora tengo " + toString(this.edad) + " años.";
}
}
```

// --- Clase derivada ---

```
class Estudiante : Persona {
    let grado: integer;

    function constructor(nombre: string, edad: integer, grado: integer) {
        // No hay 'super': inicializamos campos heredados directamente
        this.nombre = nombre;
        this.edad = edad;
        this.color = "rojo";
        this.grado = grado;
    }

    function estudiar(): string {
        return this.nombre + " está estudiando en " + toString(this.grado) + "
grado.";
    }

    function promedioNotas(nota1: integer, nota2: integer, nota3:
integer): integer {
```

```
    let promedio: integer = (nota1 + nota2 + nota3) / 3; // división entera
    return promedio;
}
}
```

```
// --- Programa principal ---
```

```
let log: string = "";
```

```
let nombre: string = "Erick";
```

```
let juan: Estudiante = new Estudiante(nombre, 20, 3);
```

```
// "Imprimir" = concatenar al log con saltos de línea
```

```
log = log + juan.saludar() + "\n";
```

```
log = log + juan.estudiar() + "\n";
```

```
log = log + juan.incrementarEdad(5) + "\n";
```

```
// Bucle (uso de while por compatibilidad)
```

```
let i: integer = 1;
```

```
while (i <= 5) {
```

```
    if ((i % 2) == 0) {
```

```
        log = log + toString(i) + " es par\n";
```

```
    } else {
```

```
        log = log + toString(i) + " es impar\n";
```

```
}  
  
i = i + 1;  
  
}
```

// Expresión aritmética (entera)

```
let resultado: integer = (juan.edad * 2) + ((5 - 3) / 2);
```

```
log = log + "Resultado de la expresión: " + toString(resultado) + "\n";
```

// Ejemplo de promedio (entero)

```
let prom: integer = 0;
```

```
prom = juan.promedioNotas(90, 85, 95);
```

```
log = log + "Promedio (entero): " + toString(prom) + "\n";
```

// Nota: 'log' contiene todas las salidas.

Salida del compilador, código MIPS

.data

str_0: .asciiz ""

str_1: .asciiz "Hola, mi nombre es "

str_2: .asciiz "Ahora tengo "

str_3: .asciiz " años."

str_4: .asciiz " está estudiando en "

str_5: .asciiz " grado."

str_6: .asciiz "Erick"

str_7: .asciiz "\n"

str_8: .asciiz " es par\n"

str_9: .asciiz " es impar\n"

str_10: .asciiz "Resultado de la expresión: "

str_11: .asciiz "Promedio (entero): "

.text

.globl main

func_toString:

--- Prologue for toString ---

addi \$sp, \$sp, -128

sw \$ra, 0(\$sp)

sw \$fp, 4(\$sp)

move \$fp, \$sp

main:

--- Prologue for main ---

addi \$sp, \$sp, -256

```

sw $ra, 0($sp)

sw $fp, 4($sp)

move $fp, $sp

# TAC no reconocido: ActivationRecord toString

sw $a0, -8($fp)    # p_x = LoadParam 0

# TAC no reconocido: .frame

# TAC no reconocido: .param p_x, [bp+4]

# TAC no reconocido: .endframe

la $t0, str_0

sw $t0, -12($fp)    # t1 = ""

lw $v0, -12($fp)    # Set return value from t1

# --- Epilogue for main ---

lw $ra, 0($fp)

lw $fp, 4($fp)

addi $sp, $sp, 256

jr $ra

# TAC no reconocido: EndFunc toString

func_constructor:

# --- Prologue for constructor ---

addi $sp, $sp, -128

sw $ra, 0($sp)

sw $fp, 4($sp)

move $fp, $sp

# TAC no reconocido: ActivationRecord constructor

sw $a0, -8($fp)    # p_nombre = LoadParam 0

sw $a1, -12($fp)    # p_edad = LoadParam 1

```

```

# TAC no reconocido: .frame

# TAC no reconocido: .param p_nombre, [bp+4]

# TAC no reconocido: .param p_edad, [bp+8]

# TAC no reconocido: .endframe

lw $t0, -16($fp)

sw $t0, -16($fp)  # nombre = nombre

lw $t0, -20($fp)

sw $t0, -20($fp)  # edad = edad

lw $t0, -24($fp)

sw $t0, -24($fp)  # color = color

# --- Epilogue for constructor ---

lw $ra, 0($fp)

lw $fp, 4($fp)

addi $sp, $sp, 128

jr $ra

# TAC no reconocido: EndFunc constructor

func_saludar:

# --- Prologue for saludar ---

addi $sp, $sp, -128

sw $ra, 0($sp)

sw $fp, 4($sp)

move $fp, $sp

# TAC no reconocido: ActivationRecord saludar

la $t0, str_1

sw $t0, -8($fp)  # t1 = "Hola, mi nombre es "

li $t0, 0

```

```

sw $t0, -12($fp)  # PLACEHOLDER for t2 = getprop this, nombre
lw $t0, -8($fp)   # Load t1
lw $t1, -12($fp)  # Load t2
add $t0, $t0, $t1  # t1 = t1 + t2
sw $t0, -8($fp)   # Store result in t1
lw $v0, -8($fp)   # Set return value from t1
# --- Epilogue for saludar ---
lw $ra, 0($fp)
lw $fp, 4($fp)
addi $sp, $sp, 128
jr $ra

# TAC no reconocido: EndFunc saludar

func_incrementarEdad:
# --- Prologue for incrementarEdad ---
addi $sp, $sp, -128
sw $ra, 0($sp)
sw $fp, 4($sp)
move $fp, $sp

# TAC no reconocido: ActivationRecord incrementarEdad
sw $a0, -8($fp)   # p_anos = LoadParam 0
# TAC no reconocido: .frame
# TAC no reconocido: .param p_anos, [bp+4]
# TAC no reconocido: .endframe
lw $t0, -12($fp)
sw $t0, -12($fp)  # edad = edad
la $t0, str_2

```

```

sw $t0, -16($fp)  # t1 = "Ahora tengo "
li $t0, 0

sw $t0, -20($fp)  # PLACEHOLDER for t2 = this.edad
lw $a0, -20($fp)  # Load param t2 into $a0
jal func_toString  # t2 = call toString, 1
sw $v0, -20($fp)  # t2 = return value
lw $t0, -16($fp)  # Load t1
lw $t1, -20($fp)  # Load t2
add $t0, $t0, $t1  # t1 = t1 + t2
sw $t0, -16($fp)  # Store result in t1
la $t0, str_3

sw $t0, -20($fp)  # t2 = " años."
lw $t0, -16($fp)  # Load t1
lw $t1, -20($fp)  # Load t2
add $t0, $t0, $t1  # t1 = t1 + t2
sw $t0, -16($fp)  # Store result in t1
lw $v0, -16($fp)  # Set return value from t1

# --- Epilogue for incrementarEdad ---
lw $ra, 0($fp)
lw $fp, 4($fp)
addi $sp, $sp, 128
jr $ra

# TAC no reconocido: EndFunc incrementarEdad

func_constructor:

# --- Prologue for constructor ---
addi $sp, $sp, -128

```

```

sw $ra, 0($sp)

sw $fp, 4($sp)

move $fp, $sp

# TAC no reconocido: ActivationRecord constructor

sw $a0, -8($fp)  # p_nombre = LoadParam 0

sw $a1, -12($fp) # p_edad = LoadParam 1

sw $a2, -16($fp) # p_grado = LoadParam 2

# TAC no reconocido: .frame

# TAC no reconocido: .param p_nombre, [bp+4]

# TAC no reconocido: .param p_edad, [bp+8]

# TAC no reconocido: .param p_grado, [bp+12]

# TAC no reconocido: .endframe

lw $t0, -20($fp)

sw $t0, -20($fp) # nombre = nombre

lw $t0, -24($fp)

sw $t0, -24($fp) # edad = edad

lw $t0, -28($fp)

sw $t0, -28($fp) # color = color

lw $t0, -32($fp)

sw $t0, -32($fp) # grado = grado

# --- Epilogue for constructor ---

lw $ra, 0($fp)

lw $fp, 4($fp)

addi $sp, $sp, 128

jr $ra

# TAC no reconocido: EndFunc constructor

```

func_estudiar:

--- Prologue for estudiar ---

addi \$sp, \$sp, -128

sw \$ra, 0(\$sp)

sw \$fp, 4(\$sp)

move \$fp, \$sp

TAC no reconocido: ActivationRecord estudiar

li \$t0, 0

sw \$t0, -8(\$fp) # PLACEHOLDER for t1 = getprop this, nombre

la \$t0, str_4

sw \$t0, -12(\$fp) # t2 = " está estudiando en "

lw \$t0, -8(\$fp) # Load t1

lw \$t1, -12(\$fp) # Load t2

add \$t0, \$t0, \$t1 # t1 = t1 + t2

sw \$t0, -8(\$fp) # Store result in t1

li \$t0, 0

sw \$t0, -12(\$fp) # PLACEHOLDER for t2 = this.grado

lw \$a0, -12(\$fp) # Load param t2 into \$a0

jal func_toString # t2 = call toString, 1

sw \$v0, -12(\$fp) # t2 = return value

lw \$t0, -8(\$fp) # Load t1

lw \$t1, -12(\$fp) # Load t2

add \$t0, \$t0, \$t1 # t1 = t1 + t2

sw \$t0, -8(\$fp) # Store result in t1

la \$t0, str_5

sw \$t0, -12(\$fp) # t2 = " grado."

```

lw $t0, -8($fp)    # Load t1
lw $t1, -12($fp)   # Load t2
add $t0, $t0, $t1   # t1 = t1 + t2
sw $t0, -8($fp)     # Store result in t1
lw $v0, -8($fp)     # Set return value from t1
# --- Epilogue for estudiar ---
lw $ra, 0($fp)
lw $fp, 4($fp)
addi $sp, $sp, 128
jr $ra

# TAC no reconocido: EndFunc estudiar

func_promedioNotas:
# --- Prologue for promedioNotas ---
addi $sp, $sp, -128
sw $ra, 0($sp)
sw $fp, 4($sp)
move $fp, $sp

# TAC no reconocido: ActivationRecord promedioNotas
sw $a0, -8($fp)     # p_nota1 = LoadParam 0
sw $a1, -12($fp)    # p_nota2 = LoadParam 1
sw $a2, -16($fp)    # p_nota3 = LoadParam 2
# TAC no reconocido: .frame
# TAC no reconocido: .param p_nota1, [bp+4]
# TAC no reconocido: .param p_nota2, [bp+8]
# TAC no reconocido: .param p_nota3, [bp+12]
# TAC no reconocido: .endframe

```

```

lw $t0, -24($fp)
sw $t0, -20($fp)  # t1 = nota1
lw $t0, -32($fp)
sw $t0, -28($fp)  # t2 = nota2
lw $t0, -20($fp)  # Load t1
lw $t1, -28($fp)  # Load t2
add $t0, $t0, $t1  # t1 = t1 + t2
sw $t0, -20($fp)  # Store result in t1
lw $t0, -36($fp)
sw $t0, -28($fp)  # t2 = nota3
lw $t0, -20($fp)  # Load t1
lw $t1, -28($fp)  # Load t2
add $t0, $t0, $t1  # t1 = t1 + t2
sw $t0, -20($fp)  # Store result in t1
li $t0, 3
sw $t0, -28($fp)  # t2 = 3
lw $t0, -20($fp)  # Load t1
lw $t1, -28($fp)  # Load t2
div $t0, $t1      # t1 = t1 / t2
mflo {self.temp_reg_pool[0]}
sw $t0, -20($fp)  # Store result in t1
lw $t0, -40($fp)
sw $t0, -28($fp)  # t2 = promedio
lw $v0, -28($fp)  # Set return value from t2
# --- Epilogue for promedioNotas ---
lw $ra, 0($fp)

```

```

lw $fp, 4($fp)

addi $sp, $sp, 128

jr $ra

# TAC no reconocido: EndFunc promedioNotas

la $t0, str_0

sw $t0, -28($fp)  # t2 = ""

la $t0, str_6

sw $t0, -44($fp)  # t3 = "Erick"

li $t0, 3

sw $t0, -48($fp)  # t4 = 3

lw $a0, -48($fp)  # Load param t4 into $a0

li $t0, 20

sw $t0, -48($fp)  # t4 = 20

lw $a1, -48($fp)  # Load param t4 into $a1

lw $t0, -52($fp)

sw $t0, -48($fp)  # t4 = nombre

lw $a2, -48($fp)  # Load param t4 into $a2

jal func_newEstudiante # t4 = call newEstudiante, 3

sw $v0, -48($fp)  # t4 = return value

lw $t0, -60($fp)

sw $t0, -56($fp)  # t5 = log

lw $a0, -64($fp)  # Load param juan into $a0

jal func_saludar  # t6 = call method saludar, 1

sw $v0, -68($fp)  # t6 = return value

lw $t0, -56($fp)  # Load t5

lw $t1, -68($fp)  # Load t6

```

```

add $t0, $t0, $t1  # t5 = t5 + t6

sw $t0, -56($fp)  # Store result in t5

la $t0, str_7

sw $t0, -68($fp)  # t6 = "\n"

lw $t0, -56($fp)  # Load t5

lw $t1, -68($fp)  # Load t6

add $t0, $t0, $t1  # t5 = t5 + t6

sw $t0, -56($fp)  # Store result in t5

lw $t0, -56($fp)

sw $t0, -60($fp)  # log = t5

lw $t0, -60($fp)

sw $t0, -56($fp)  # t5 = log

lw $a0, -64($fp)  # Load param juan into $a0

jal func_estudiar  # t6 = call method estudiar, 1

sw $v0, -68($fp)  # t6 = return value

lw $t0, -56($fp)  # Load t5

lw $t1, -68($fp)  # Load t6

add $t0, $t0, $t1  # t5 = t5 + t6

sw $t0, -56($fp)  # Store result in t5

la $t0, str_7

sw $t0, -68($fp)  # t6 = "\n"

lw $t0, -56($fp)  # Load t5

lw $t1, -68($fp)  # Load t6

add $t0, $t0, $t1  # t5 = t5 + t6

sw $t0, -56($fp)  # Store result in t5

lw $t0, -56($fp)

```

```

sw $t0, -60($fp)  # log = t5
lw $t0, -60($fp)
sw $t0, -56($fp)  # t5 = log
li $t0, 5
sw $t0, -68($fp)  # t6 = 5
lw $a0, -68($fp)  # Load param t6 into $a0
lw $a1, -64($fp)  # Load param juan into $a1
jal func_incrementarEdad # t6 = call method incrementarEdad, 2
sw $v0, -68($fp)  # t6 = return value
lw $t0, -56($fp)  # Load t5
lw $t1, -68($fp)  # Load t6
add $t0, $t0, $t1  # t5 = t5 + t6
sw $t0, -56($fp)  # Store result in t5
la $t0, str_7
sw $t0, -68($fp)  # t6 = "\n"
lw $t0, -56($fp)  # Load t5
lw $t1, -68($fp)  # Load t6
add $t0, $t0, $t1  # t5 = t5 + t6
sw $t0, -56($fp)  # Store result in t5
lw $t0, -56($fp)
sw $t0, -60($fp)  # log = t5
li $t0, 1
sw $t0, -56($fp)  # t5 = 1
L1:
lw $t0, -72($fp)
sw $t0, -68($fp)  # t6 = i

```

```

li $t0, 5
sw $t0, -76($fp)  # t7 = 5
lw $t0, -68($fp)
lw $t1, -76($fp)
sle $t0, $t0, $t1  # t6 = t6 <= t7
sw $t0, -68($fp)
lw $t0, -68($fp)
beq $t0, $zero, L2  # if t6 == 0 goto L2
lw $t0, -72($fp)
sw $t0, -68($fp)  # t6 = i
li $t0, 2
sw $t0, -76($fp)  # t7 = 2
lw $t0, -68($fp)  # Load t6
lw $t1, -76($fp)  # Load t7
div $t0, $t1      # t6 = t6 % t7
mfhi $t0
sw $t0, -68($fp)  # Store result in t6
li $t0, 0
sw $t0, -76($fp)  # t7 = 0
lw $t0, -68($fp)
lw $t1, -76($fp)
seq $t0, $t0, $t1  # t6 = t6 == t7
sw $t0, -68($fp)
lw $t0, -68($fp)
beq $t0, $zero, L3  # if t6 == 0 goto L3
lw $t0, -60($fp)

```

```

sw $t0, -68($fp)  # t6 = log
lw $t0, -72($fp)
sw $t0, -76($fp)  # t7 = i
lw $a0, -76($fp)  # Load param t7 into $a0
jal func_toString # t7 = call toString, 1
sw $v0, -76($fp)  # t7 = return value
lw $t0, -68($fp)  # Load t6
lw $t1, -76($fp)  # Load t7
add $t0, $t0, $t1 # t6 = t6 + t7
sw $t0, -68($fp)  # Store result in t6
la $t0, str_8
sw $t0, -76($fp)  # t7 = " es par\n"
lw $t0, -68($fp)  # Load t6
lw $t1, -76($fp)  # Load t7
add $t0, $t0, $t1 # t6 = t6 + t7
sw $t0, -68($fp)  # Store result in t6
lw $t0, -68($fp)
sw $t0, -60($fp)  # log = t6
j L4             # goto L4

```

L3:

```

lw $t0, -60($fp)
sw $t0, -68($fp)  # t6 = log
lw $t0, -72($fp)
sw $t0, -76($fp)  # t7 = i
lw $a0, -76($fp)  # Load param t7 into $a0
jal func_toString # t7 = call toString, 1

```

```

sw $v0, -76($fp)  # t7 = return value
lw $t0, -68($fp)  # Load t6
lw $t1, -76($fp)  # Load t7
add $t0, $t0, $t1  # t6 = t6 + t7
sw $t0, -68($fp)  # Store result in t6
la $t0, str_9
sw $t0, -76($fp)  # t7 = " es impar\n"
lw $t0, -68($fp)  # Load t6
lw $t1, -76($fp)  # Load t7
add $t0, $t0, $t1  # t6 = t6 + t7
sw $t0, -68($fp)  # Store result in t6
lw $t0, -68($fp)
sw $t0, -60($fp)  # log = t6

```

L4:

```

lw $t0, -72($fp)
sw $t0, -68($fp)  # t6 = i
li $t0, 1
sw $t0, -76($fp)  # t7 = 1
lw $t0, -68($fp)  # Load t6
lw $t1, -76($fp)  # Load t7
add $t0, $t0, $t1  # t6 = t6 + t7
sw $t0, -68($fp)  # Store result in t6
lw $t0, -68($fp)
sw $t0, -72($fp)  # i = t6
j L1             # goto L1

```

L2:

```

li $t0, 0

sw $t0, -68($fp)  # PLACEHOLDER for t6 = getprop juan, edad

li $t0, 2

sw $t0, -76($fp)  # t7 = 2

lw $t0, -68($fp)  # Load t6

lw $t1, -76($fp)  # Load t7

sub $t0, $t0, $t1  # t6 = t6 * t7

sw $t0, -68($fp)  # Store result in t6

li $t0, 5

sw $t0, -76($fp)  # t7 = 5

li $t0, 3

sw $t0, -80($fp)  # t8 = 3

lw $t0, -76($fp)  # Load t7

lw $t1, -80($fp)  # Load t8

sub $t0, $t0, $t1  # t7 = t7 - t8

sw $t0, -76($fp)  # Store result in t7

li $t0, 2

sw $t0, -80($fp)  # t8 = 2

lw $t0, -76($fp)  # Load t7

lw $t1, -80($fp)  # Load t8

div $t0, $t1      # t7 = t7 / t8

mflo {self.temp_reg_pool[0]}

sw $t0, -76($fp)  # Store result in t7

lw $t0, -68($fp)  # Load t6

lw $t1, -76($fp)  # Load t7

add $t0, $t0, $t1  # t6 = t6 + t7

```

```

sw $t0, -68($fp)  # Store result in t6

lw $t0, -60($fp)

sw $t0, -76($fp)  # t7 = log

la $t0, str_10

sw $t0, -80($fp)  # t8 = "Resultado de la expresión: "

lw $t0, -76($fp)  # Load t7

lw $t1, -80($fp)  # Load t8

add $t0, $t0, $t1  # t7 = t7 + t8

sw $t0, -76($fp)  # Store result in t7

lw $t0, -84($fp)

sw $t0, -80($fp)  # t8 = resultado

lw $a0, -80($fp)  # Load param t8 into $a0

jal func_toString  # t8 = call toString, 1

sw $v0, -80($fp)  # t8 = return value

lw $t0, -76($fp)  # Load t7

lw $t1, -80($fp)  # Load t8

add $t0, $t0, $t1  # t7 = t7 + t8

sw $t0, -76($fp)  # Store result in t7

la $t0, str_7

sw $t0, -80($fp)  # t8 = "\n"

lw $t0, -76($fp)  # Load t7

lw $t1, -80($fp)  # Load t8

add $t0, $t0, $t1  # t7 = t7 + t8

sw $t0, -76($fp)  # Store result in t7

lw $t0, -76($fp)

sw $t0, -60($fp)  # log = t7

```

```

li $t0, 0

sw $t0, -76($fp)  # t7 = 0

li $t0, 95

sw $t0, -80($fp)  # t8 = 95

lw $a0, -80($fp)  # Load param t8 into $a0

li $t0, 85

sw $t0, -80($fp)  # t8 = 85

lw $a1, -80($fp)  # Load param t8 into $a1

li $t0, 90

sw $t0, -80($fp)  # t8 = 90

lw $a2, -80($fp)  # Load param t8 into $a2

lw $a3, -64($fp)  # Load param juan into $a3

jal func_promedioNotas # t8 = call method promedioNotas, 4

sw $v0, -80($fp)  # t8 = return value

lw $t0, -80($fp)

sw $t0, -88($fp)  # prom = t8

lw $t0, -60($fp)

sw $t0, -80($fp)  # t8 = log

la $t0, str_11

sw $t0, -92($fp)  # t9 = "Promedio (entero): "

lw $t0, -80($fp)  # Load t8

lw $t1, -92($fp)  # Load t9

add $t0, $t0, $t1  # t8 = t8 + t9

sw $t0, -80($fp)  # Store result in t8

lw $t0, -88($fp)

sw $t0, -92($fp)  # t9 = prom

```

```
lw $a0, -92($fp)  # Load param t9 into $a0
jal func_toString # t9 = call toString, 1
sw $v0, -92($fp)  # t9 = return value
lw $t0, -80($fp)  # Load t8
lw $t1, -92($fp)  # Load t9
add $t0, $t0, $t1 # t8 = t8 + t9
sw $t0, -80($fp)  # Store result in t8
la $t0, str_7
sw $t0, -92($fp)  # t9 = "\n"
lw $t0, -80($fp)  # Load t8
lw $t1, -92($fp)  # Load t9
add $t0, $t0, $t1 # t8 = t8 + t9
sw $t0, -80($fp)  # Store result in t8
lw $t0, -80($fp)
sw $t0, -60($fp)  # log = t8
```